

Final Project Summary — Architecture-Aware CUDA NTT on the NVIDIA RTX 2080 Ti

Scope. This is the final, self-contained summary of the project’s core contribution: the architecture-aware CUDA implementation and optimization of the Number Theoretic Transform (NTT) on an NVIDIA RTX 2080 Ti, benchmarked head-to-head against the GPU-NTT library on the same GPU. Earlier Python/PyTorch/Triton/C++ implementations appear only as preliminary development baselines; the Kyber/ML-KEM work is mentioned once as an application outlook.

Authoritative data. Every number in this document traces to a committed raw CSV from the final execution-verified run (SLURM **job 153**, node `srvr-2`), with the selector, correctness, and stream-level results from jobs 151–153. Where an earlier run of the same grid (job 151: 49/7/18, max 2.14×) differs at boundary cells, the job-153 numbers are used throughout and the run-to-run spread is stated explicitly. Superseded intermediate maps (e.g. the 56-cell job-139 map under a 10%-margin selector) are not quoted here.

Key raw files: `results/raw/final_cuda_vs_existing.csv` · `final_cuda_backend_map.csv` · `final_cuda_output_order_cost.csv` · `final_cuda_selector_execution.csv` · `final_cuda_representative_throughput.csv` · `final_cuda_summary_table.csv` · `final_cuda_kernel_audit.csv` · `final_cuda_hardware_profile.csv` · `final_cuda_energy.csv` · `final_ntt_optimization_progression.csv` · `results/raw/final_cuda/ptxas_verbose.csv`

Final figures: `results/plots/final_cuda_summary/` (regenerated from the CSVs above by `src/plot_final_cuda.py`)

1. Project objective

Implement the forward cyclic NTT (modulus $q = 998244353$, primitive root 3, $N = 256 \dots 16384$, batch $1 \dots 8192$) as specialized CUDA kernels for the RTX 2080 Ti, understand per-regime bottlenecks from measurements, optimize each regime with an architecture-specific technique, and compare honestly against a reproduced external CUDA NTT library (GPU-NTT) on the same GPU, same modulus, same timing harness, with correctness gated before every timing.

The project began as a Python $\rightarrow$ C++ $\rightarrow$ PyTorch $\rightarrow$ Triton study; those stages established the methodology (batching, twiddle caching, kernel-count reduction) but were 6–32× slower than the first fused CUDA kernel. Everything below is the CUDA phase.

2. Background: NTT on a GPU

The NTT is the FFT over $\mathbb{Z}_q$: $X[k] = \sum x[n] \cdot \omega^{\{nk\}} \bmod q$ with ω an N -th root of unity. A radix-2 Cooley-Tukey NTT is $\log_2 N$ butterfly stages; each butterfly is one modular multiply plus a modular add/sub. On a GPU the practical bottlenecks are (a) the integer modular-multiply dependency chain, (b) shared-memory bank conflicts in the strided exchange between stages, (c) block-level synchronization once data no longer fits a warp, (d) grid occupancy when one transform maps to one block, and (e) DRAM bandwidth once batches are large. **No single kernel is optimal across (N, batch)** because the binding constraint changes with the regime — this observation drives the whole design (Section 5) and is why the final system is a set of specialized kernels behind a measured dispatch table (Section 6).

3. Hardware platform (Table 1)

Measured on the benchmark node via `cudaGetDeviceProperties` (`tools/cuda_hw_profile/hw_profile.cu`, `results/raw/final_cuda_hardware_profile.csv`).

property	value	consequence for the NTT design
GPU / architecture	NVIDIA GeForce RTX 2080 Ti, Turing TU102, compute capability 7.5	no <code>cp.async (sm_80+)</code> , no tensor-core integer path used — plain-SIMT INT32 design
SMs	68	one block per transform uses 1/68 SMs at batch=1 → multi-block 4-step kernel
CUDA cores	4352 (68 SM × 64 FP32). Turing adds a separate, concurrent 64-wide INT32 datapath per SM — the 4352 figure counts FP32 cores, <i>not</i> “4352 INT32 cores”	modular butterflies are integer IMAD chains on the INT32 units; FP32 units are idle; INT32 upper bound ≈6.7 T op/s at 1545 MHz is <i>analytically derived</i> , never claimed as achieved
warp	32 lanes, 32 resident warps/SM (1024 threads)	5 butterfly stages (m=1...16) fit inside one warp → <code>__shfl_xor_sync</code> exchange with zero shared memory and zero barriers
register file	64 K registers/SM	≤32 regs/thread keeps 100% theoretical occupancy; Shoup’s extra precomputed operand fits “for free” (22-29 regs)
shared memory / L1	64 KB/SM usable, unified 96 KB with L1	N=16384×4 B = 64 KB is the single-block ceiling → padded kernel capped at N≤8192; 32 banks → padding rule SPAD(i)=i+[i/32]
L2 cache	5.5 MB	twiddle tables (≤64 KB) stay L2-resident → only the 8 B/coefficient data traffic matters at scale
memory	GDDR6, 352-bit → 616 GB/s theoretical	the DRAM roof; saturated small-N cells measured ≈37-39% of it
clocks / toolchain	1545 MHz boost; driver 550.120, CUDA 12.4, nvcc -O3 -std=c++17 -gencode arch=compute_75,code=sm_75	timings after warm-up at steady clocks

Metric provenance (Table 8, used throughout):

metric	status
kernel latency (CUDA events), p50/p95/p99, energy (NVML counter, workload mixes)	measured
registers/thread, spills, static shared memory	compiler-reported (ptxas -v)
theoretical occupancy, transforms/s, coefficients/s, % of 616 GB/s	analytically derived
effective DRAM bandwidth (known bytes ÷ measured time)	estimated — <i>not</i> a profiler counter and <i>not</i> “GPU utilization”
achieved occupancy, SM-active %, eligible warps/cycle, stall reasons	unavailable — no Nsight Compute / nvprof on this cluster; binding regimes were attributed by controlled experiments instead

4. Initial CUDA bottlenecks

The first fused single-kernel uint32 CUDA NTT (bit-reversal folded into the shared-memory load) closed the 8-30× gap to GPU-NTT down to ~1.1-1.3× at batch=128, and measurement isolated four distinct residual bottlenecks, each owned by a different regime:

1. **Small N (≤1024):** latency-bound — the modular-multiply dependency chain plus the launch floor dominate (time is nearly flat in batch up to ~64).
2. **Mid N (2048-8192):** shared-memory bound — bank conflicts in the strided cross-stage exchange plus log N `__syncthreads()` barriers (arithmetic changes were experimentally neutral here).
3. **Large N at low batch:** grid-occupancy-bound — one 1024-thread block per transform runs on 1 of 68 SMs (measured worst case 4.31× slower than GPU-NTT at N=16384, batch=1).
4. **Any N at saturation:** DRAM-bound — nothing arithmetic moves it; both our kernels and GPU-NTT plateau near ~30-39% of the theoretical 616 GB/s.

5. The CUDA kernel families (Table 2)

kernel family	source	regs (ptxas)	spills	target regime	key idea
warp-shuffle (+Shoup)	tools/low_latency_cuda_ntt/warp_shuffle_0ntt_kernels.cu, tools/shoup_cuda_ntt/shoup_shuffle_kernels.cu	10h22	0	N≤1024, batch up to ~2048-4096	5 register-resident butterfly stages via <code>__shfl_xor_sync</code>
padded shared-memory (+Shoup)	tools/shared_memory_0ntt/27-29 tools/shoup_cuda_ntt/shoup_padded_kernels.cu	84t	0	N=2048-8192 mid/high batch	bank-conflict-free layout SPAD(i)=i+[i/32]
multi-block 4-step	tools/multiblock_cuda_ntt/37multiblock_ntt0kernels.cu	45ntt	37	N=4096-16384, batch 1-8	N=N ₁ ·N ₂ split across 128+ blocks

kernel family	source	regs (ptxas)	spills	target regime	key idea
low-latency / fused (retained)	tools/low_latency_cuda-42t/ tools/competitive_cuda_ntt/	26-42t	0	dominated — fallback lineage	pair-per-thread shared-memory butterflies
GPU-NTT (external backend)	external/GPU-NTT (Merge NTT)	—	—	saturated regions	per-stage streaming kernels

All selected kernels: **zero register spills** (ptxas-verified, final_cuda/ptxas_verbose.txt), 19-45 registers/thread, one kernel launch per batched transform set (multi-block: two).

5.1 Warp-shuffle kernel (small N)

One block per transform, one element per thread (nthreads = N, valid for $N \leq 1024$). In a decimation-in-time NTT, stage st pairs elements at distance $m = 2^{st}$. For $m < 32$ both butterfly operands live in the **same warp**, so lane t 's partner is lane $t \oplus m$ and the operand is exchanged **in registers** with `__shfl_xor_sync(0xffffffff, reg, m)` — the first five stages ($m = 1, 2, 4, 8, 16$) run with **no shared-memory traffic, no bank conflicts, and no `__syncthreads()`**. Each lane computes the add or the subtract side of the butterfly depending on $t \& m$. Stages with $m \geq 32$ are cross-warp and fall back to shared memory with one barrier per stage — so a full $N=256$ transform pays only $\log_2 N - 5 = 3$ barriers instead of 8. Bit-reversal is folded into the initial load (`__brev`), so output is natural-order at no extra pass.

Why it wins: small N is latency-bound; removing the shared-memory round-trip and most barriers attacks exactly the critical path. Measured: up to **1.93× over GPU-NTT** ($N=256$, batch 1-2, job 153; 2.14× in the job-151 run), latency floor 4.5 μ s, win region holding $>1.05\times$ to batch $\approx 2048-4096$ at $N=256$.

Why it eventually loses: at saturation ($N=256$ batch 8192) the cell is DRAM/issue-bound; the register tricks no longer matter and GPU-NTT's per-stage streaming is $\sim 5\%$ faster (0.0699 vs 0.0739 ms) — the runtime hands off there.

5.2 Padded shared-memory kernel (mid N)

For $N = 2048-8192$ the transform is staged entirely in shared memory (pair-per-thread, up to 1024 threads). Turing shared memory has **32 banks of 4-byte words**; a strided butterfly exchange at distance m makes threads of one warp hit addresses congruent modulo 32, serializing the accesses (bank conflicts). The fix is one padding word per 32 elements:

```
SPAD(i) = i + floor(i / 32)          // shared_mem_ntt_kernels.cu: ((i) + ((i) >> 5))
```

The pad shifts every 32-element group by one word, so the strided accesses of a warp land in **distinct banks** across all stages, at the cost of $\sim 3\%$ more shared memory (`SPAD_WORDS(N)`). ptxas confirms 0 spills at ≤ 34 registers. Measured effect of the padding alone (same kernel, padded vs unpadded indexing, `shared_memory_optimization_benchmark.csv`): up to **1.23× at $N=1024$, 1.27× at $N=2048$, 1.32× at $N=4096$, 1.29× at $N=8192$** .

5.3 Shoup modular multiplication (arithmetic on the critical chain)

The butterfly multiply $(u64)a \cdot b \% q$ compiles to a Barrett-style sequence of $\sim 4-5$ dependent integer ops on the critical path of every stage. Every butterfly multiplier is a *twiddle* known at table-build time, so the Shoup/Harvey trick precomputes the companion $w' = \text{floor}(w \cdot 2^{32} / q)$ per twiddle; at runtime:

```
q̂ = __umulhi(w', x); r = w·x - q̂·q; if (r ≥ q) r -= q;    // all 32-bit
```

Correctness follows the standard Shoup bound ($r \in [0, 2q)$, and $2q < 2^{32}$ for $q = 998244353$); validated bit-exact against the non-Shoup kernels at every (N , batch). The cost is a **doubled twiddle table** (uint2) and a specialized kernel variant per family — which is why Shoup was applied **only to the two kernels the runtime actually selects** (warp-shuffle and padded); applying it to a non-dominant

kernel was measured to change nothing at the system level. Measured gain in individual cells: up to $\approx 1.20\times$ (shoup_shuf vs best non-Shoup at $N=256$ batch 1, adaptive_selector_shoup_dominant.csv; typically $1.05\text{--}1.12\times$). The 64 K-register file absorbs the extra operand: 22 regs (shuffle) / 27–29 regs (padded) at 100% theoretical occupancy.

5.4 Multi-block 4-step NTT (large N, low batch)

A single-block $N \geq 2048$ kernel uses 1024 threads = one SM’s whole thread budget, so at batch=1 the other 67 SMs idle — the measured origin of the $4.31\times$ worst-case gap. The 4-step Cooley-Tukey decomposition factors $N = N_1 \cdot N_2$ (e.g. 128×128 at $N=16384$) and runs:

1. N_1 independent N_2 -point sub-NTTs (N_1 blocks),
2. a twiddle multiply by $\omega_N^{\{n_1 k_2\}}$ (fused),
3. N_2 independent N_1 -point sub-NTTs (N_2 blocks).

One transform becomes **$N_1 + N_2 \geq 128$ blocks of 64–128 threads**, filling all 68 SMs even at batch=1 (up to 16 blocks/SM). The index mapping is a bijection to natural order, so **no final transpose pass** is required; the cost is **two kernel launches and roughly twice the global traffic** (two read+write passes plus a scattered write). Measured vs the prior single-block best (final_ntt/final_multiblock.csv, job 105/106 series confirmed by the final rerun): **$1.49\times$ at $N=4096$ batch=1, $2.25\times$ at $N=8192$ batch=1, $3.60\times$ at $N=16384$ batch=1, $1.66\times$ at $N=16384$ batch=8** (final rerun values $1.66/2.25/3.77/1.72$ — same direction, boundary noise). This cut the $N=16384$ batch=1 gap vs GPU-NTT from $4.31\times$ to $\approx 1.0\text{--}1.16\times$ (0.97 , i.e. parity, in job 153). The extra traffic makes it **lose at high batch** ($0.49\text{--}0.73\times$), so the selector only routes batches 1–8 to it.

5.5 Adaptive selector (the deployable system)

The selector is a **compiled, executed dispatch layer** (tools/adaptive_ntt_runtime/adaptive_runtime.cu), *not* an offline argmin over a spreadsheet:

- **Mechanism:** a per-cell table keyed on $(\log_2 N - 8, \log_2 \text{batch})$ holds the measured-best backend; runK indexes it and launches through one switch. Measured lookup overhead: **1.25 ns** (amortized over 10^7 lookups) against a $\geq 4.5 \mu\text{s}$ cheapest transform — negligible.
- **Unseen (N, batch):** requests map to their $\log_2$ bucket (nearest measured cell); values outside the grid fall back to the region rule (small- $N \rightarrow$ shuffle + Shoup, mid $\rightarrow$ padded, large- N low-batch $\rightarrow$ multi-block, else GPU-NTT). Documented behavior, not extrapolated performance.
- **Execution-verified (job 153):** for each of the 74 cells the *selected* backend was actually launched (REP=100) after selection; output **values and ordering both pass 74/74** (final_cuda_selector_execution.csv). **Regret** (executed p50 $\div$ separately measured best): median **0.998**, mean 0.997, **worst 1.051** ($N=2048$ batch=16) — the median sits slightly below 1 because the “best” table and the execution run are separate runs with independent noise. The worst case means **“never worse than the best backend” must not be claimed:** the honest statement is *worst observed slowdown $\approx 5.1\%$, median at parity*.
- **GPU-specific by construction:** the table is measured on THIS RTX 2080 Ti; the mechanism ports, the numbers do not — a different GPU requires re-profiling.
- **Stream-level caveat:** on a large- N -dominated heterogeneous mix, backend switching costs launch pipelining — pure GPU-NTT dispatch is $\sim 7\%$ faster than adaptive on that mix, while adaptive is $1.59\times$ faster than our-kernels-only. Energy on the same mix (NVML total-energy counter, two-point subtraction): gpu_only 0.178, adaptive 0.192, our_only 0.232 $\mu\text{J}/\text{NTT}$.

6. Experimental methodology

Head-to-head against **GPU-NTT** (Özcan & Savaş “Merge NTT”, github.com/Alisah-Ozcan/GPU-NTT @95c739c), built from source on the same node, same sm_75 flags, modulus pinned to 998244353 (documented edit). Both sides: compute-only (data GPU-resident, twiddles prebuilt), CUDA-event timed, WARM 3–10 / REP 9–30 medians, correctness gated *before* timing on edge

inputs (zeros, one-hot, all-(q-1), random $\times 2$) plus inverse round trip, $N=256-16384$ — all pass. The transform convention was **formally verified**: our natural-order output is bit-exact with the direct DFT at $\omega = 3^{((q-1)/N)}$ and equals the bit-reversal permutation of GPU-NTT's output (tools/final_correctness/convention_diag.cu).

Grid: $N \in \{256 \dots 16384\}$; batch 1...128 for all N , extended to 8192 for $N \leq 1024 \rightarrow$ **74 cells**.

Why no other external systems appear in the numbers (final_cuda_baseline_methodology.md): GPU-NTT is the **only external CUDA NTT reproduced experimentally on this GPU at this transform scope**. Excluded, with reasons: **ICICLE** — GPU NTT backend closed-source, small-field build pinned to babybear (2013265921), not 998244353; **cuPQC** — distributed for sm_80+ only, cannot run on this sm_75 card; **HI-Kyber** and **Tensor-Core Kyber** works — no buildable same-scope artifact for this GPU/modulus, and tensor-core INT paths are a different arithmetic model. Their published numbers come from different GPUs/fields and are **not** quoted as if measured here.

7. Correctness and the output-order tradeoff

Our kernels emit **natural-order** output natively (bit-reversal folded into the load; the 4-step mapping is order-preserving). GPU-NTT emits **bit-reversed** output natively. Both compute the same transform values (formally verified). A bitrev_permute kernel was measured at all 74 cells (one extra global read+write — a large fraction of a transform, e.g. 0.0346 ms vs the 0.0739 ms transform at $N=256$ $B=8192$), giving three comparison contracts (Table 6, final_cuda_output_order_cost.csv, ties = ratio within $\pm 3\%$):

output contract	our wins	within 3%	GPU-NTT wins	max our speedup
native layout (each side its own order — primary)	48	4	22	1.93×
natural order (GPU-NTT pays the permutation)	69	2	3	2.75×
bit-reversed (we pay the permutation)	3	6	65	1.09×

The native-layout and common-output comparisons **answer different questions**. Native layout is the fair setting for self-consistent NTT $\rightarrow$ pointwise-multiply $\rightarrow$ INTT pipelines, where each library's inverse consumes its own forward order and no conversion is ever needed — all headline numbers use it. The two normalized contracts are the sensitivity bounds for consumers that demand a specific layout: natural-order output is a genuine *feature* of our kernels (a consumer needing it gets it free, from GPU-NTT only at the permutation price), and symmetrically **GPU-NTT is much stronger when bit-reversed output is the desired interface**. The runtime's public API is an explicit per-call layout flag that charges the permutation to whichever backend mismatches.

8. Results vs GPU-NTT (native layout, job 153)

48 wins / 4 within 3% / 22 GPU-NTT wins over 74 cells; maximum speedup 1.93× ($N=256$, batch 1-2). Max per N : 1.93× (256), 1.62× (512), 1.48× (1024), 1.38× (2048), 1.34× (4096), 1.18× (8192); at $N=16384$ the best is 0.97 (parity, batch=1). The strongest region is **$N \leq 1024$ at batches 1-2048** — the latency-critical and mid-throughput regime of lattice-crypto-shaped workloads. GPU-NTT wins the saturated cells ($N \leq 1024$ at batch ≥ 4096 and $N=16384$ at mid/high batch), where its per-stage streaming kernels are DRAM-optimal; the selector routes those cells to GPU-NTT. Run-to-run: job 151 on the same grid gave 49/7/18 and max 2.14× — boundary cells flip within $\pm 3\%$ noise; sub-10 μ s small- N latencies vary $\sim 20\%$ with clock state, direction stable.

Main optimization gains (Table 3)

optimization	measured gain	vs what	where
warp-shuffle execution	up to 1.61× (0.62 ratio)	GPU-NTT	$N \leq 1024$ small batch (pre-Shoup)
+ Shoup butterfly	up to $\approx 1.20\times$ (typ. 1.05–1.12×)	same kernel non-Shoup	selected kernels only
shared-memory padding	1.23× / 1.27× / 1.32× / 1.29×	same kernel unpadded	$N=1024$ / 2048 / 4096 / 8192
multi-block 4-step	1.49× / 2.25× / 3.60× / 1.66×	prior single-block best	$N=4096$ B1 / 8192 B1 / 16384 B1 / 16384 B8

optimization	measured gain	vs what	where
argmin selector (vs 10%-margin)	7 boundary cells recovered, up to 1.146×	margin selector	N=4096-8192
combined (system)	48/74 wins, max 1.93×	GPU-NTT	native layout

Representative performance (Table 4 — results/raw/final_cuda_summary_table.csv)

workload	N	batch	backend	batch ms	μs/NTT	NTT/s	coeff/s	est. GB/s	% of 616	vs GPU-NTT	scope
small-N low batch	256	1	shuffle + Shoup	0.00458	4.58	0.219 M	5.6e7	0.4	0.1%	1.93×	native
small-N pre-saturation	256	2048	shuffle + Shoup	0.0213	0.010	96.2 M	2.5e10	197	32%	1.08×	native
small-N saturation (handoff)	256	8192	GPU-NTT	0.0699	0.009	117.2 M	3.0e10	240	39%	0.95	native
— our-kernel peak, same cell	256	8192	shuffle + Shoup	0.073856	0.009	110.9 M	2.8e10	227	36.9%	0.95	native
medium-N	2048	32	padded + Shoup	0.00896	0.280	3.57 M	7.3e9	59	9.5%	1.33×	native
large-N low batch	16384	1	GPU-NTT	0.01024	10.24	98 K	1.6e9	13	2.1%	0.97	native
large-N high batch	16384	128	GPU-NTT	0.0897	0.701	1.43 M	2.3e10	187	30%	0.82	native

Every throughput headline carries its (N, batch): the peak our-kernel figure is **110.9 million NTT/s at N=256, batch=8192** (warp_shuffle_shoup, 0.073856 ms/batch, raw natural-order output, ≈ 227 GB/s $\approx 37\%$ of the 616 GB/s theoretical peak) — and at that saturated cell **GPU-NTT is $\sim 5\%$ faster (117.2 M NTT/s)**, so the runtime selects GPU-NTT there. The effective-bandwidth percentages are traffic-based estimates, **not GPU utilization**.

Selector overhead and regret (Table 7)

quantity	value	status
lookup overhead	1.25 ns/dispatch	measured (10 ⁷ -lookup amortized)
executed-vs-best regret, median / mean	0.998 / 0.997	measured (74 cells, REP=100)
worst observed regret	1.051 ($\approx 5.1\%$ slower, N=2048 B=16)	measured — “never worse” is NOT claimed
executed correctness (values + ordering)	74/74 PASS	measured
stream-level (large-N-heavy mix D)	adaptive 7% behind gpu-only, 1.59× ahead of ours-only	measured
energy (mix D, NVML)	0.192 μJ/NTT adaptive (0.178 gpu-only, 0.232 ours-only)	measured

9. Tradeoffs and limitations

- **Saturation handoff:** our single-kernel designs win until DRAM saturation, then lose (0.79–0.96× at $N \leq 1024$ B=8192; 0.49–0.88× at $N=16384$ mid/high batch); the selector’s job is to know that boundary from measurement.
- **Output order is a contract, not a free win:** we are strong under native and natural-order contracts, weak (3/74) under a bit-reversed contract.
- **Selector is not oracle-perfect:** worst observed 5.1% regret; table is specific to this GPU.
- **Specialization cost:** Shoup doubles twiddle storage and forks kernel variants; multi-block doubles global traffic; each is deployed only in its measured win region.
- **Profiling gaps:** no Nsight Compute on the cluster — achieved occupancy, SM-active %, eligible warps/cycle, and stall breakdowns were never measured; bottleneck attributions rest on controlled experiments (batch-scaling flatness, arithmetic-neutrality tests, block-count scaling) and

are labeled as such.

- **Baseline scope:** one external library reproduced; claims cover nothing that could not be run.

10. Final contribution

On an NVIDIA RTX 2080 Ti, we developed specialized CUDA NTT kernels using warp-shuffle communication, Shoup modular multiplication, conflict-aware shared-memory layouts, and multi-block 4-step decomposition. Across 74 execution-verified native-layout configurations, our kernels outperform GPU-NTT in 48 cases, are within 3% in four, and reach a maximum speedup of approximately $1.93\times$. The strongest region is $N \leq 1024$ at batches 1–2048. Output layout is an explicit feature and tradeoff: under a natural-order contract our implementation wins 69 of 74 configurations, whereas GPU-NTT dominates when bit-reversed output is requested. A compiled table-driven selector chooses the appropriate backend with approximately 1.25 ns lookup overhead and a worst observed slowdown of about 5.1% relative to the separately measured best backend.

Not claimed: global state of the art; always faster than GPU-NTT; per-cell never worse; any utilization figure (only estimated effective bandwidth); measured achieved occupancy; coverage of external systems that could not be reproduced.

Application outlook (one paragraph, outside the core claim): the same principles (transform fusion, NTT-domain accumulation, measured dispatch) were exercised on Kyber’s $q=3329$ incomplete NTT as a case study with a $12.3\times$ arithmetic-core improvement over its isolated-transform baseline, and full ML-KEM pipeline routing/scheduling experiments exist as verified future-work material; none of it is part of the CUDA NTT claim above.

Figures (all in results/plots/final_cuda_summary/, generated by src/plot_final_cuda_s

figure	shows	source CSV
speedup_vs_gpu_ntt_heatmap.png	per-cell speedup vs GPU-NTT, diverging around $1\times$; blue = we win (small/mid N, low/mid batch), red = GPU-NTT (saturation)	final_cuda_vs_existing.csv
backend_selection_map.png	which backend the argmin selector runs per cell — the measured regime structure	final_cuda_backend_map.csv
transforms_per_second.png	throughput scaling in batch per N, runtime-selected backend; saturation knees visible	final_cuda_backend_map.csv
output_contract_win_breakdown.png	48/4/22 vs 69/2/3 vs 3/6/65 — the contract decides the comparison	final_cuda_output_order_cost.csv
optimization_progression.png	per-stage measured improvement, each vs its own stated baseline (not cumulative)	final_ntt_optimization_progression.csv
effective_bandwidth_regimes.png	estimated effective bandwidth (% of 616 GB/s) per representative regime — explicitly not utilization	final_cuda_representative_throughput.csv

Figure gallery

Figure 1 — Per-cell speedup vs GPU-NTT. Diverging around 1×; blue = our kernels win (small/mid N, low/mid batch), red = GPU-NTT wins (saturation). Source: final_cuda_vs_existing.csv.

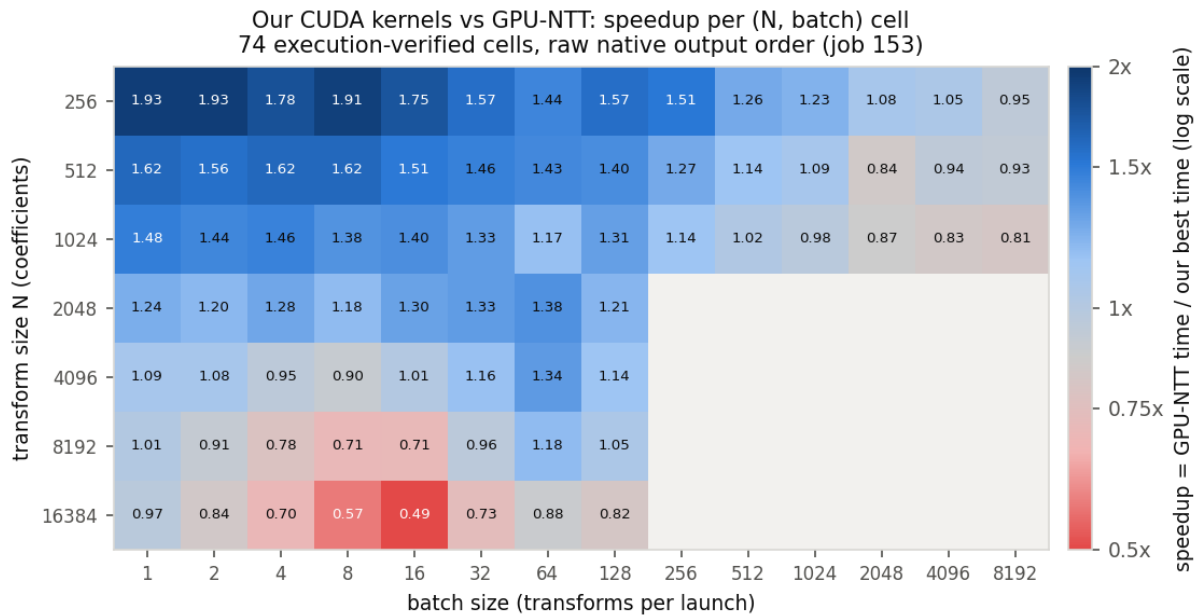


Figure 1: Per-cell speedup vs GPU-NTT

Figure 2 — Backend selection map. Which backend the argmin selector runs per cell — the measured regime structure. Source: `final_cuda_backend_map.csv`.

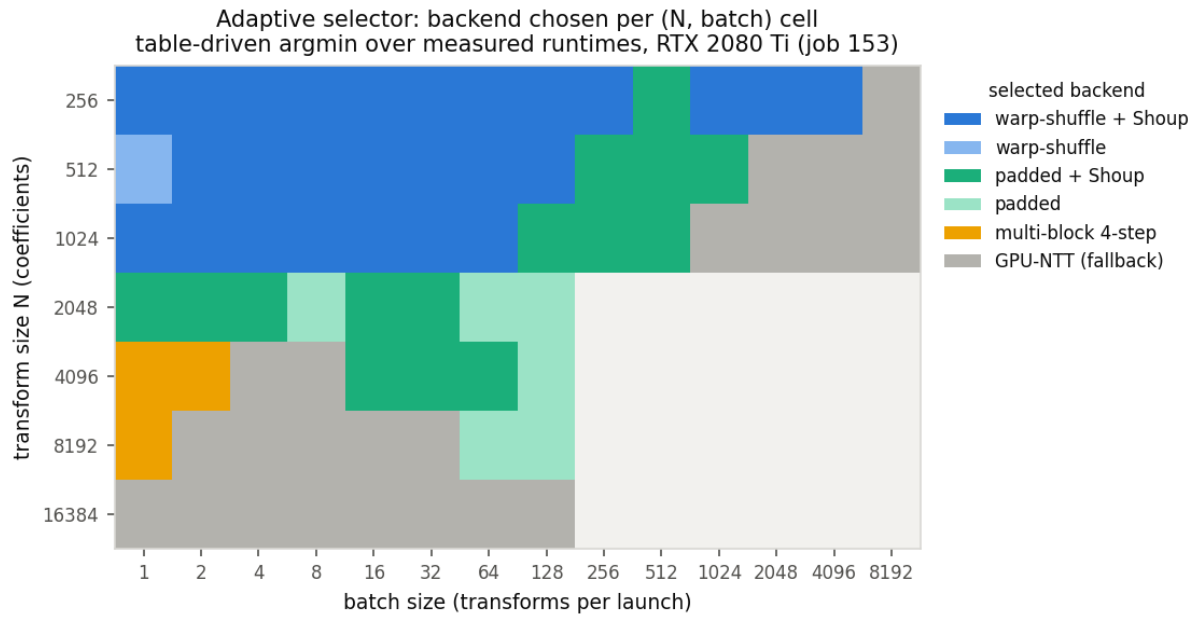


Figure 2: Backend selection map

Figure 3 — Transforms per second. Throughput scaling in batch per N, runtime-selected backend; saturation knees visible. Source: final_cuda_backend_map.csv.

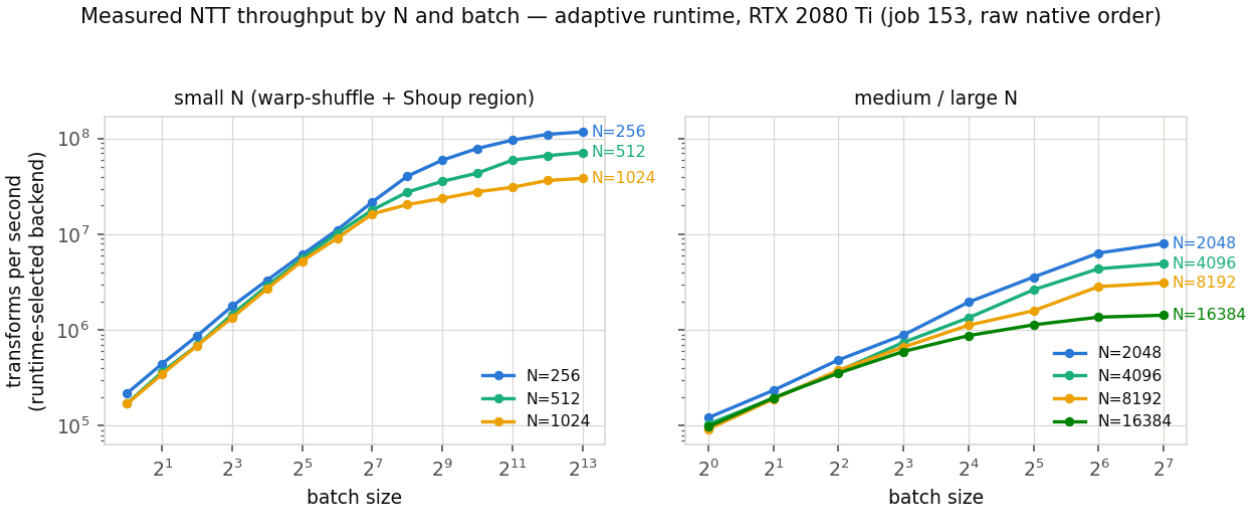


Figure 3: Transforms per second

Figure 4 — Output-contract win breakdown. 48/4/22 vs 69/2/3 vs 3/6/65 — the contract decides the comparison. Source: final_cuda_output_order_cost.csv.

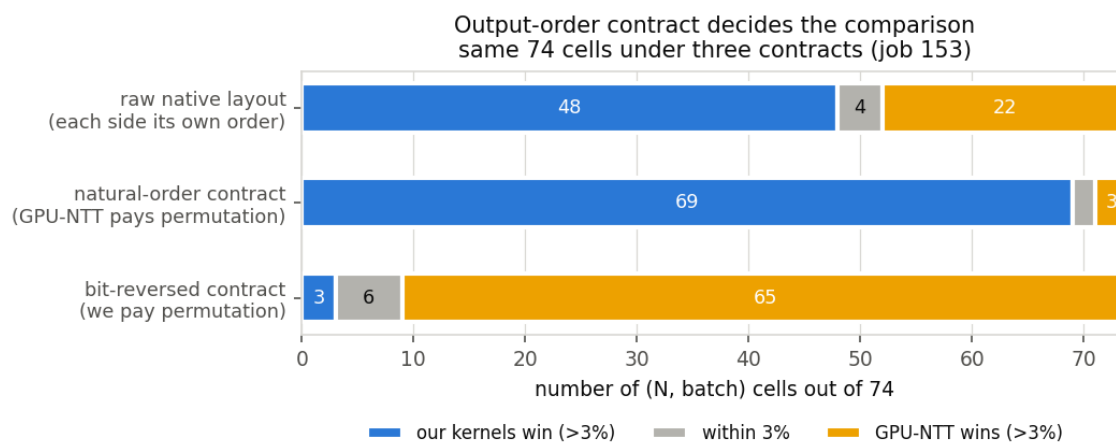


Figure 4: Output contract win breakdown

Figure 5 — Optimization progression. Per-stage measured improvement, each vs its own stated baseline (not cumulative). Source: final_ntt_optimization_progression.csv.

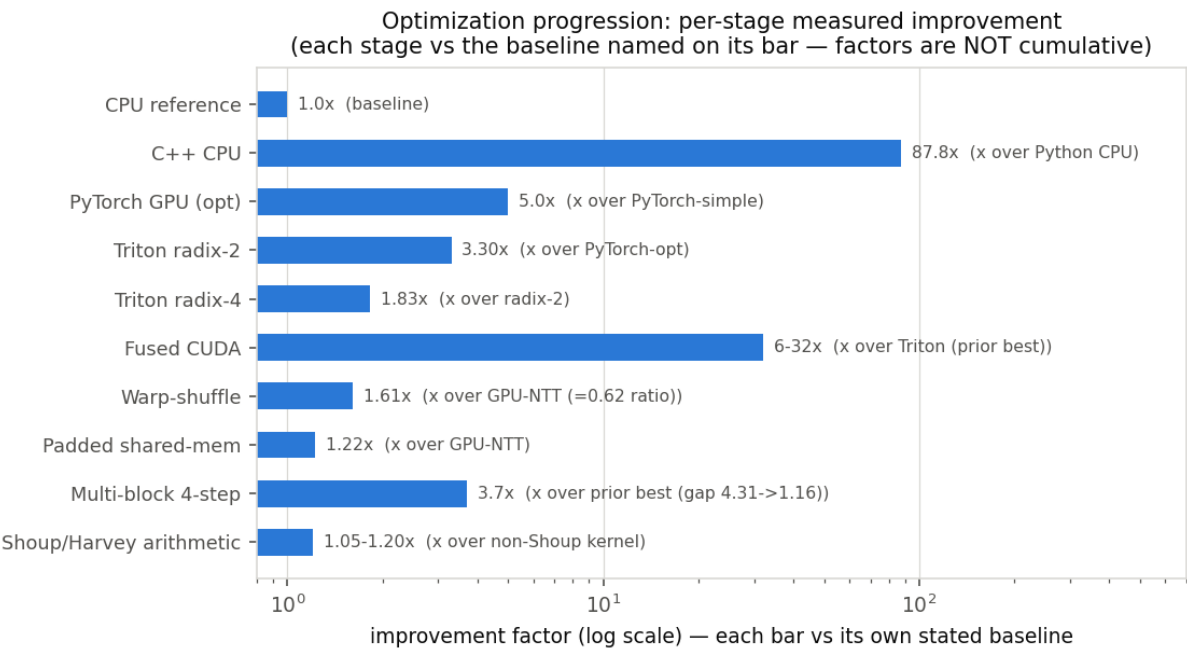


Figure 5: Optimization progression

Figure 6 — Effective bandwidth regimes. Estimated effective bandwidth (% of 616 GB/s) per representative regime — explicitly not utilization. Source: final_cuda_representative_throughput.csv.

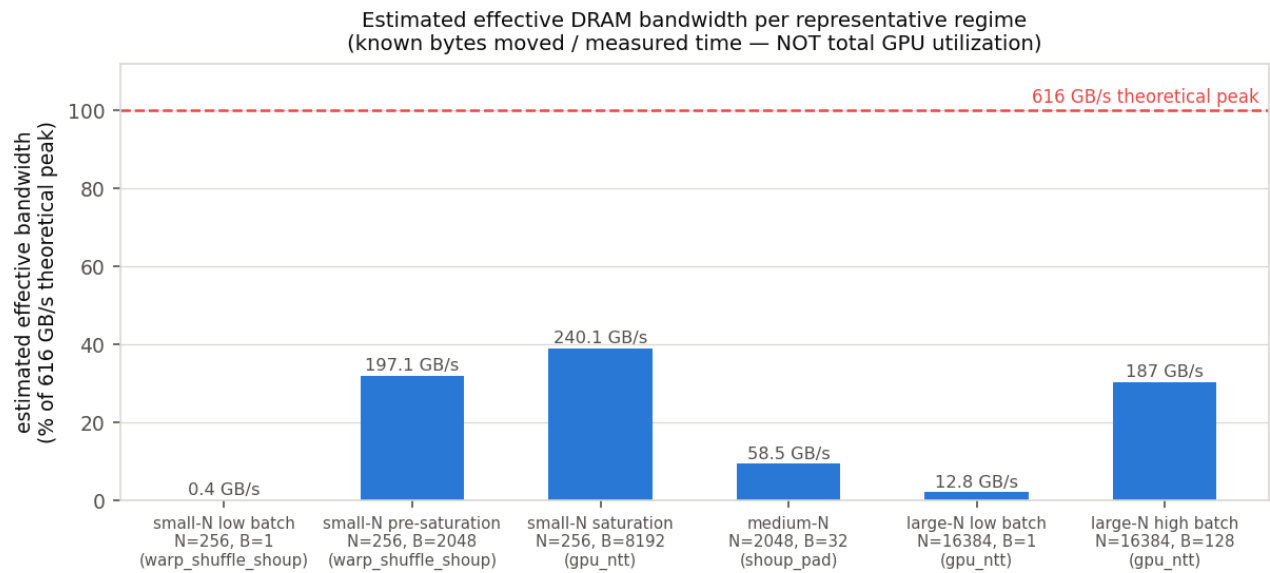


Figure 6: Effective bandwidth regimes